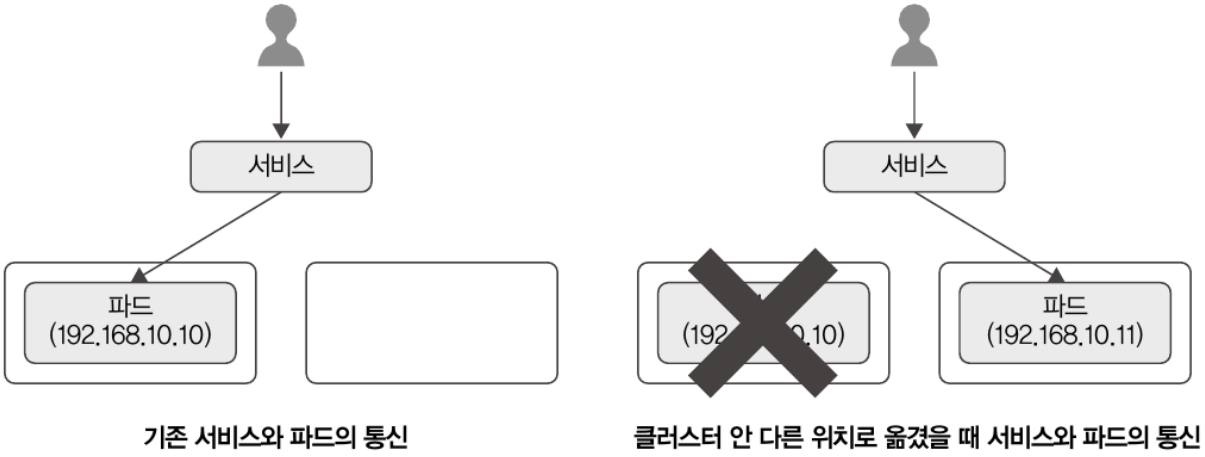
Ch7. 서비스

서비스는 **여러 개 파드에 접근할 수 있는 IP 하나를 제공**한다. 다양한 기능을 제공하지만 본질적으로 로드밸런서 역할이다.

1. 서비스의 개념

- **서비스**: **동적으로 변하는 파드들에 고정적으로 접근**할 때 사용하는 방법
  - 서비스를 사용하면 파드가 클러스터 안 어디에 있든 고정 주소를 이용해 접근할 수 있다.
  - 클러스터 외부에서 클러스터 안 파드에 접근할 수 있다.
  - 인그레스로도 접근할 수 있는데 서비스는 주로 L4 영역에서 통신할 때 사용하고 인그레스는 L7 영역에서 접근할 때 사용한다는 차이점이 있다.

그림 7-1 서비스를 이용한 파드와의 통신 원리



2. 서비스 타입

ClusterIP

- 기본 서비스 타입이며 **쿠버네티스 클러스터 안에서 사용**할 수 있다.
- 클러스터 안 노드나 파드에서는 클러스터 IP를 이용해서 서비스에 연결된 파드에 접근
- 클러스터 외부에서는 이용할 수 없다.

NodePort

- **서비스 하나에 모든 노드의 지정된 포트를 할당**한다.
- **클러스터 안 뿐만 아니라 클러스터 외부에서도 접근**할 수 있다.
- 클러스터 외부에서 클러스터 안 파드로 접근할 때 사용할 수 있는 가장 간단한 방법

LoadBalancer

- 아마존 웹서비스, 구글 클라우드 플랫폼 같은 **퍼블릭 클라우드 서비스**, 오픈스택 같은 프라이빗 클라우드, 쿠버네티스를 지원하는 로드밸런서 장비에서 사용
- 클라우드에서 제공하는 로드밸런서와 파드를 연결한 수 해당 로드밸런서의 IP를 이용해 클러스터 외부에서 파드에 접근할 수 있도록 한다.
- kubectl get service 명령으로 서비스 상태를 확인하면 **EXTERNAL-IP** 항목에 로드밸런서 IP를 표시

ExternalName

- 서비스를 .spec.externalName 필드에 설정한 값과 연결한다.
- **클러스터 안에서 외부에 접근**할 때 주로 사용
- 클러스터 외부에 접근하면 **설정해둔 CNAME 값을 이용**해 클러스터 외부에 접근할 수 있다.

---

3. 서비스 사용하기

- 서비스의 기본 설정 예(service/service.yaml)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP
  clusterIP: 10.0.10.10
  selector:
    app: MyApp
  ports:
```

```
  - protocol: TCP
    port: 80
    targetPort: 9376
```

- .spec.type 필드: 서비스 타입 지정. 설정하지 않으면 ClusterIP이다.
- .spec.ClusterIP 필드에서 클러스터 IP를 직접 설정할 수 있다. 설정하지 않으면 자동으로 IP값이 할당
- .spec.selector 필드에는 서비스와 연결할 파드에 설정한 .label 필드 값을 설정
- .spec.ports[] 필드는 배열 형태

```
$ kubectl run nginx-pod --image=nginx --port=80 --labels=ap
p=MyApp
```

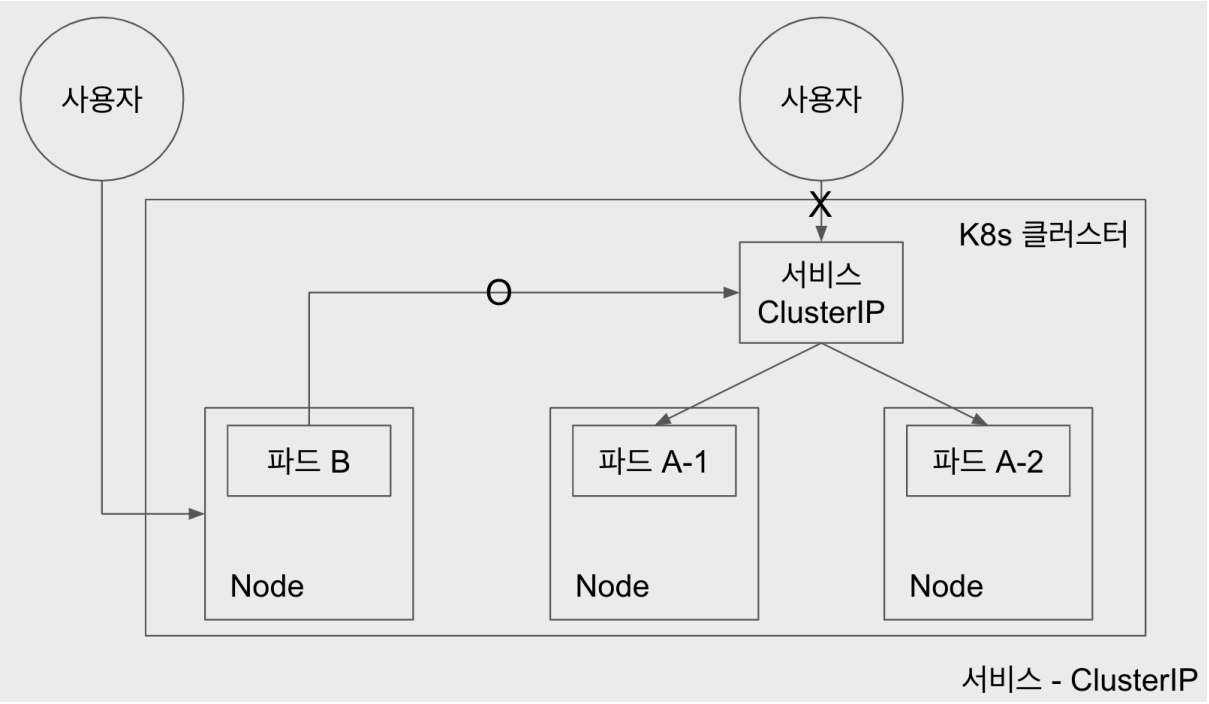
3.1 ClusterIP 타입 서비스 사용하기

- ClusterIP 타입의 서비스 설정 예(service/clusterip.yaml)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: clusterip-service
spec:
  type: ClusterIP
  selector:
    app: nginx-for-svc
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
```

- .spec.type 필드 값: ClusterIP
- .spec.selector.app 필드 값: nginx-for-svc

- ClusterIP 타입 서비스의 전체 구조



- kubectl apply -f clusterip.yaml
- kubectl get svc
  - CLUSTER-IP 항목에 클러스터 IP가 생성
  - EXTERNAL-IP 항목은 외부 IP가 없으므로 <none>
  - PORT(S)에는 80 포트가 연결 정보

```
$ kubectl get svc
NAME                TYPE        CLUSTER-IP     EXTERNAL-IP
PORT(S)   AGE
clusterip-service   ClusterIP   10.102.93.83   <none>
80/TCP    12s
```

- **kubectl describe service 서비스명**
  - 좀 더 자세한 정보를 확인

```
$ kubectl describe service clusterip-service
Name:               clusterip-service
Namespace:          default
Labels:             <none>
Annotations:        <none>
Selector:           app=nginx-for-svc
```

```
Type:              ClusterIP
IP Family Policy:  SingleStack
IP Families:       IPv4
IP:                10.102.93.83
IPs:               10.102.93.83
Port:              <unset>  80/TCP
TargetPort:        80/TCP
Endpoints:         10.32.0.2:80
Session Affinity:  None
Events:            <none>
$ kubectl get po -o wide
NAME        READY   STATUS    RESTARTS   AGE   IP
NODE          NOMINATED NODE   READINESS GATES
nginx-pod   1/1     Running   0          22h   10.32.0.2
k8s-worker2   <none>           <none>
```

- 앤드포인트에 접근할 수 있는지 확인
  - 클러스터 IP는 쿠버네티스 클러스터 안에서만 사용할 수 IP이므로 쿠버네티스 클러스터 안에 파드를 하나 실행하고 해당 파드를 통해 접속을 시도
- **kubectl run -it** testnet --image nicolaka/netshoot bash

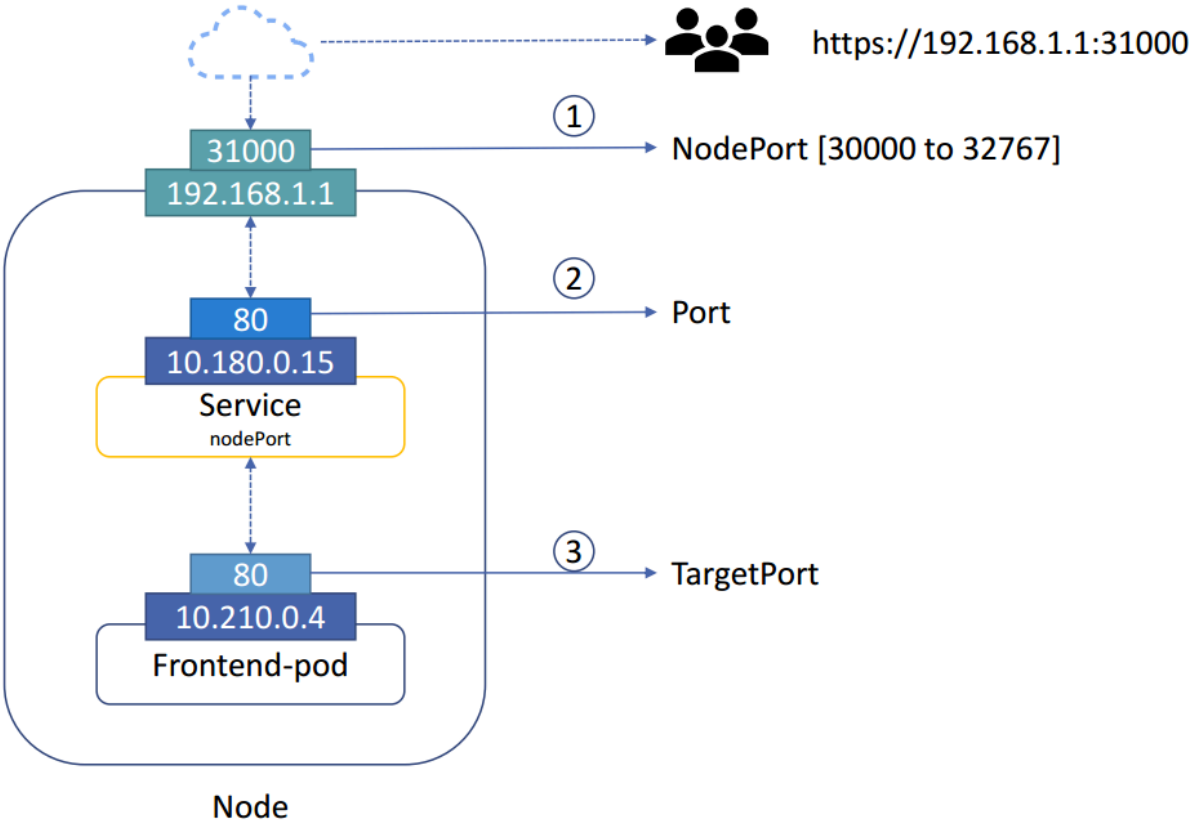
```
testnet:~# curl 10.32.0.2
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
...
</html>
```

3.2 NodePort 타입 서비스 사용하기

- NodePort 타입 서비스 예(service/nodeport.yaml)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nodeport-service
spec:
  type: NodePort
  selector:
    app: nginx-for-svc
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
    nodePort: 30080
```

- NodePort 타입 서비스의 전체 구조



- ClusterIP 타입 서비스처럼 쿠버네티스 클러스터 안에 따로 파드를 만들어 접속할 필요가 없다. 바로 웹 브라우저를 열고 localhost:30080으로 접속해도 nginx 웹 페이지가 보인다.
- kubectl apply -f nodeport.yaml

```
$ kubectl get svc
NAME                 TYPE        CLUSTER-IP      EXTERNAL-IP
PORT(S)        AGE
kubernetes           ClusterIP   10.96.0.1       <none>
443/TCP        42s
nodeport-service     NodePort    10.110.223.77   <none>
80:30080/TCP   7s
```

- NodePort 타입이지만 CLUSTER-IP 항목이 10.110.223.77로 설정되었음을 확인

```
$ curl localhost:30080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
...
</html>
```

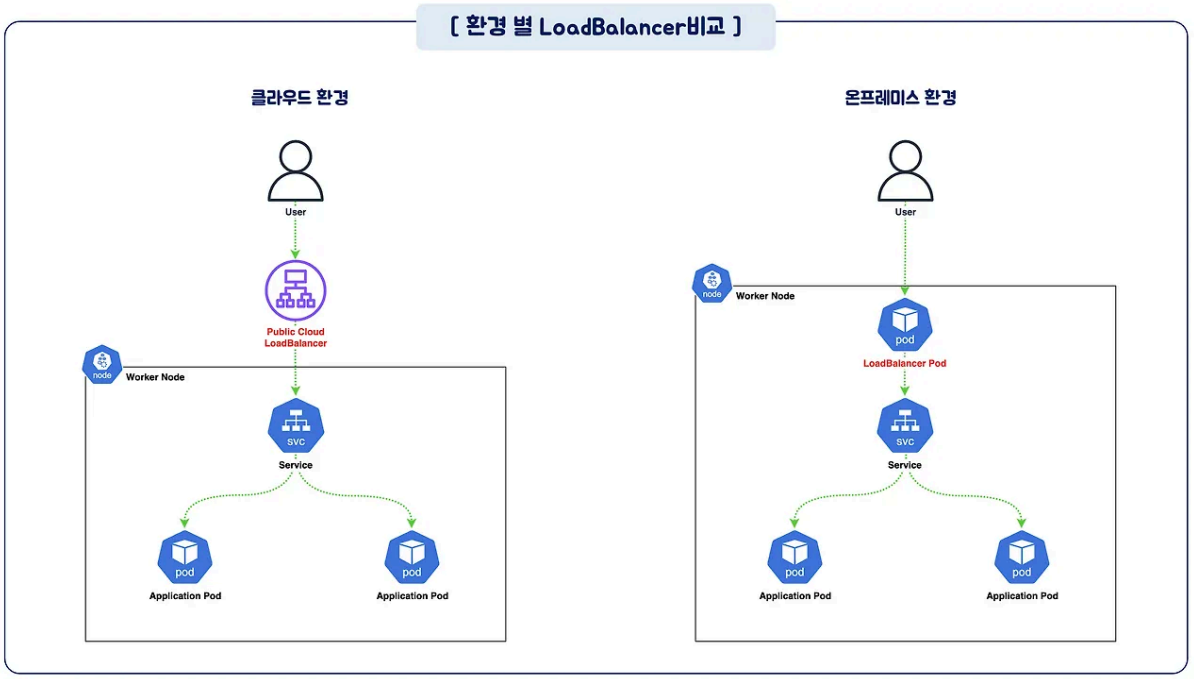
3.3 LoadBalancer 타입 서비스 사용하기

- LoadBalancer 타입 서비스 설정 예(service/loadbalnacer.yaml)

```
apiVersion: v1
kind: Service
metadata:
  name: loadbalancer-service
spec:
  type: LoadBalancer
  selector:
    app: nginx-for-svc
  ports:
  - protocol: TCP
```

```
    port: 80
    targetPort: 80
```

- 쿠버네티스 클러스터를 외부 로드밸런서와 연계해서 설치했을 때 사용
- 로드밸런서 타입 서비스의 구조



- kubectl apply -f loadbalancer.yaml

```
$ kubectl get svc
NAME                    TYPE           CLUSTER-IP     EXTERN
AL-IP   PORT(S)         AGE
kubernetes              ClusterIP      10.96.0.1      <none>
443/TCP        12s
loadbalancer-service    LoadBalancer   10.104.55.35   <pendi
ng>     80:30780/TCP    6s
```

3.4 ExternalName 타입 서비스 사용하기

- ExternalName 타입 서비스 설정 예(service/externalname.yaml)

```
apiVersion: v1
kind: Service
```

```
metadata:
  name: externalname-service
spec:
  type: ExternalName
  externalName: google.com
```

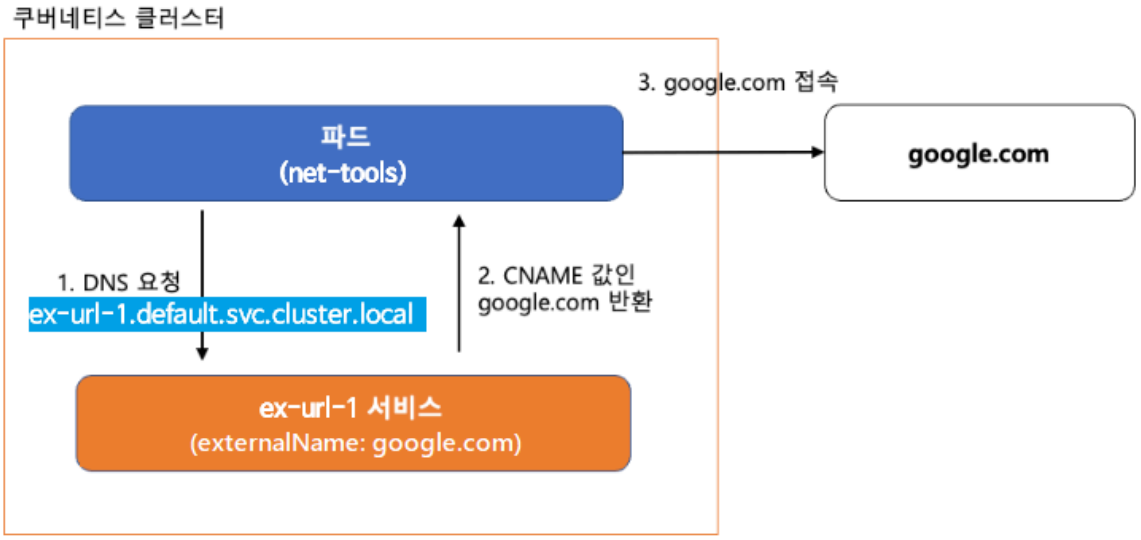
- .spec.type 필드 값으로 ExternalName 설정
- .spec.externalName 필드 값으로 google.com을 설정. 연결하려는 외부 도메인 값을 설정한 것

```
$ kubectl get svc
NAME                    TYPE           CLUSTER-IP     EXTERN
AL-IP   PORT(S)         AGE
externalname-service    ExternalName   <none>         googl
e.com    <none>          10s
```

- kubectl run -it testnet --image=nicolaka/netshoot bash

```
$ kubectl run -it testnet --image=nicolaka/netshoot -- bash
testnet:~#
```

- ExternalName 타입 서비스에서 도메인을 요청해 접속하는 구조



- kubectl run -it testnet --image=nicolaka/netshoot bash
- curl 서비스명.default.svc.cluster.local

- dig 서비스명.default.svc.cluster.local

```
testnet:~# dig externalname-service.default.svc.cluster.loc
al

; <<>> DiG 9.18.25 <<>> externalname-service.default.svc.cl
uster.local
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS quer
y is leaked to DNS
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 49454
;; flags: qr aa rd; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDI
TIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 11c60e853840beb2 (echoed)
;; QUESTION SECTION:
;externalname-service.default.svc.cluster.local.    IN A

;; ANSWER SECTION:
externalname-service.default.svc.cluster.local. 30 IN CNAME
google.com.
google.com.     30  IN  A   142.251.42.142

;; Query time: 36 msec
;; SERVER: 10.96.0.10#53(10.96.0.10) (UDP)
;; WHEN: Tue Jan 14 02:41:23 UTC 2025
;; MSG SIZE  rcvd: 183
```